

to write it.

[Why async](#)[Callbacks](#)[Promises](#)[Combinators](#)[Async/await](#)[Event loop](#)[Cheat sheet](#)

1. Why async exists

JavaScript runs your code on a single thread, one line at a time. If a slow task (network request, file read, timer) blocked that thread, the whole page would freeze. So slow tasks are handed to the browser or Node.js, and your code continues. When the task finishes, the result is delivered back to you. Callbacks, promises and async/await are three ways to receive that result.

```
// synchronous: runs in order
console.log("A");
console.log("B");

// asynchronous: C is scheduled, D runs first
setTimeout(() => console.log("C"), 1000);
console.log("D"); // prints A, B, D, then C after 1s
```

2. Callbacks

Definition: a callback is a function you pass to another function, to be called later when the work is done.

```
function getUser(id, callback) {
  setTimeout(() => {
    callback({ id, name: "Tirtho" });
  }, 500);
}

getUser(1, (user) => {
  console.log(user.name); // runs after 500ms
});
```

Error-first convention (Node.js)

The first argument is the error (or `null`), the second is the result.

```
function getUser(id, cb) {
  setTimeout(() => {
    if (id <= 0) return cb(new Error("Invalid id"));
    cb(null, { id, name: "Tirtho" });
  }, 500);
}

getUser(1, (err, user) => {
  if (err) return console.error(err.message);
  console.log(user);
});
```

The problem: callback hell

When each step depends on the previous one, callbacks nest deeper and deeper. The code drifts right, error handling is repeated at every level, and it is hard to read.

```
getUser(1, (err, user) => {
  getPosts(user.id, (err, posts) => {
    getComments(posts[0].id, (err, comments) => {
      console.log(comments); // pyramid of doom
    });
  });
});
```

Other callback risks: you hand control to another function, so it might call your callback twice, never, or too early (inversion of control).

3. Promises

Definition: a Promise is an object that represents a value that is not available yet but will be later (or will fail). It fixes nesting and gives one place for errors.

Three states

pending
work still running

fulfilled
finished, has a value

rejected
failed, has a reason

A promise settles once. After it is fulfilled or rejected, it never changes.

Creating a promise

```
function getUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id <= 0) reject(new Error("Invalid id"));
      else resolve({ id, name: "Tirtho" });
    }, 500);
  });
}
```

Consuming: then, catch, finally

```
getUser(1)
  .then(user => console.log(user.name)) // on success
  .catch(err => console.error(err.message)) // on any failure above
  .finally(() => console.log("done")); // always runs
```

Chaining (flat instead of nested)

Each `.then()` returns a new promise. Whatever you `return` becomes the input of the next step. If you return a promise, the chain waits for it.

```
getUser(1)
  .then(user => getPosts(user.id))
  .then(posts => getComments(posts[0].id))
  .then(comments => console.log(comments))
  .catch(err => console.error(err)); // one handler for the whole chain
```

Tip: always `return` inside `.then()`. Forgetting it is the most common chain bug: the next step receives `undefined`.

Shortcuts

```
Promise.resolve(42); // already fulfilled
Promise.reject(new Error("no")); // already rejected
fetch("/api/data").then(r => r.json()); // built-in APIs return promises
```

4. Running promises together

Method	Resolves when	Rejects when	Use for
<code>Promise.all</code>	all fulfill (array of values)	any one rejects	independent tasks, need all
<code>Promise.allSettled</code>	all settle (array of status objects)	never	want every outcome, even failures
<code>Promise.race</code>	first one settles	first one rejects	timeouts
<code>Promise.any</code>	first one fulfills	all reject	fastest successful source

```
const [user, posts] = await Promise.all([
  getUser(1),
  getPosts(1)
]); // runs in parallel, total time = slowest one
```

5. Async / await

Definition: syntax on top of promises that lets async code read like normal top-to-bottom code. It does not replace promises; it uses them.

Two rules

`async` before a function makes it always return a promise. `await` pauses that function until a promise settles, then gives you the value. `await` works only inside `async` functions (or at the top level of ES modules).

```
async function showComments() {
  const user = await getUser(1);
  const posts = await getPosts(user.id);
  const comments = await getComments(posts[0].id);
  console.log(comments);
}
```

```
showComments();
```

Error handling with try / catch

```
async function load() {
  try {
    const user = await getUser(1);
```

```

const user = await getUser(1),
console.log(user);
} catch (err) {
  console.error("Failed:", err.message);
} finally {
  console.log("cleanup");
}
}

```

Sequential vs parallel

```

// slow: second waits for first (about 1000ms)
const a = await wait(500);
const b = await wait(500);

// fast: both start now (about 500ms)
const [x, y] = await Promise.all([wait(500), wait(500)]);

```

Common mistakes: (1) forgetting `await`, so you get a pending Promise instead of the value; (2) using `await` inside `forEach`, which does not wait (use `for...of` or `Promise.all(arr.map(...))`); (3) awaiting independent tasks one by one.

Same task, three styles

```

// callback
getUser(1, (err, u) => { if (err) return log(err); show(u); });
// promise
getUser(1).then(show).catch(log);
// async/await
try { show(await getUser(1)); } catch (e) { log(e); }

```

6. Event loop: who runs first?

Finished async work waits in a queue until the call stack is empty. There are two queues, and the **microtask queue** (promises, `await`) is always emptied before the **macrotask queue** (`setTimeout`, events).

```

console.log("1 sync");
setTimeout(() => console.log("4 timeout"), 0);
Promise.resolve().then(() => console.log("3 promise"));
console.log("2 sync");

```

Predict the order, then run it:

Run this code

```
1 sync
2 sync
3 promise
4 timeout
```

7. Cheat sheet

	Callback	Promise	Async/await
Readability	Poor when nested	Good (flat chain)	Best (looks synchronous)
Errors	Check at each level	<code>.catch()</code>	<code>try/catch</code>
Parallel work	Manual counters	<code>Promise.all</code>	<code>await Promise.all</code>
Best for	Events, simple one-offs	Combining and chaining	Most everyday async code

Summary: callbacks are the foundation, promises add structure and error flow, async/await adds readable syntax. In modern code, write async/await, use `Promise.all` for parallel work, and always handle errors.